

Super Lab de 5 horas: Gestión del Estado y Persistencia en Microservicios sobre AKS Store Demo

Contenido

Super Lab de 5 horas: Gestión del Estado y Persistencia en
Microservicios sobre AKS Store Demo

Propósito

Duración estimada

Arquitectura base del laboratorio

Narrativa pedagógica

0. Preparación del entorno en Azure Cloud Shell

0.1 Abrir Azure Cloud Shell

0.2 Variables del laboratorio

0.3 Conectarse al AKS

0.4 Crear namespace

1. Descargar y preparar AKS Store Demo

1.1 Descargar manifiesto oficial

1.2 Evitar LoadBalancers públicos para el laboratorio

1.3 Desplegar aplicación

1.4 Validar deployments y statefulsets

2. Explorar arquitectura de microservicios

- 2.1 Ver pods
- 2.2 Ver servicios internos
- 2.3 Acceder a store-front
- 2.4 Acceder a store-admin
- 2.5 Ver logs iniciales

3. Event-driven con RabbitMQ

- 3.1 Acceder a RabbitMQ Management UI
- 3.2 Observar la cola `orders`
- 3.3 Generar actividad
- 3.4 Explicación
- 3.5 Pregunta de discusión

4. Fallo parcial: detener makeline-service

- 4.1 Escenario
- 4.2 Escalar makeline-service a cero
- 4.3 Observar RabbitMQ
- 4.4 Explicación
- 4.5 Discusión CAP
- 4.6 Restaurar makeline-service
- 4.7 Mensaje instructor

5. Fallo parcial: detener RabbitMQ

- 5.1 Escenario
- 5.2 Escalar RabbitMQ a cero
- 5.3 Revisar comportamiento
- 5.4 Pregunta para estudiantes
- 5.5 Explicación: Outbox Pattern
- 5.6 Restaurar RabbitMQ

6. Fallo parcial: detener MongoDB

- 6.1 Escenario
- 6.2 Escalar MongoDB a cero
- 6.3 Probar aplicación
- 6.4 Discusión
- 6.5 Conexión con resiliencia
- 6.6 Restaurar MongoDB

7. Redis Cache Lab

- 7.1 Objetivo
- 7.2 Desplegar Redis
- 7.3 Entrar a Redis
- 7.4 Crear producto en cache
- 7.5 Explicar Cache Aside
- 7.6 Simular stale data
- 7.7 Invalidar cache
- 7.8 Recrear cache con valor actualizado
- 7.9 Discusión de estrategias

8. CAP Theorem aplicado al laboratorio

- 8.1 Caso 1: makeline-service caído
- 8.2 Caso 2: RabbitMQ caído
- 8.3 Caso 3: MongoDB caído

9. Saga conceptual sobre AKS Store Demo

- 9.1 Flujo actual
- 9.2 Flujo Saga extendido conceptual
- 9.3 Compensaciones
- 9.4 Mensaje clave

10. Observabilidad básica

- 10.1 Ver logs por servicio
- 10.2 Seguir logs en vivo
- 10.3 Ver eventos de Kubernetes
- 10.4 Ver estado de pods
- 10.5 Mensaje clave

11. Ejercicio para estudiantes

- Escenario
- Preguntas
- Respuesta esperada

12. Checklist arquitectónico

13. Limpieza del laboratorio

- 13.1 Eliminar Redis
- 13.2 Eliminar AKS Store Demo
- 13.3 Eliminar namespace

14. Cierre para estudiantes

Propósito

Este laboratorio usa **AKS Store Demo** como aplicación base para enseñar, con una aplicación real en AKS, los principales retos de arquitectura en microservicios:

- Estado distribuido
- Persistencia en microservicios
- Mensajería asíncrona
- Eventual consistency
- Fallos parciales
- Sagas conceptuales
- Cache distribuido con Redis
- CAP Theorem aplicado
- Observabilidad básica con logs y eventos de Kubernetes
- Discusión arquitectónica: 2PC vs Sagas vs eventos

La idea no es solo “desplegar una app”, sino **romperla controladamente** para que los estudiantes entiendan qué ocurre cuando los sistemas distribuidos fallan.

Duración estimada

5 horas

Bloque	Duración
0. Preparación del entorno	20 min
1. Deploy de AKS Store Demo	30 min
2. Exploración de arquitectura	35 min
3. Event-driven con RabbitMQ	40 min
4. Fallo parcial: makeline-service	40 min
5. Fallo parcial: RabbitMQ	35 min
6. Fallo parcial: MongoDB	30 min
7. Redis Cache Lab	40 min
8. CAP + Sagas + discusión guiada	30 min
9. Checklist, conclusiones y limpieza	20 min

Arquitectura base del laboratorio

Usaremos el repositorio:

```
https://github.com/Azure-Samples/aks-store-demo
```

La aplicación tiene estos componentes principales:

Componente	Rol
store-front	Frontend para clientes
store-admin	UI administrativa para empleados
order-service	Servicio para colocar órdenes
product-service	Servicio para productos
makeline-service	Procesa órdenes desde RabbitMQ
virtual-customer	Simula creación de órdenes
virtual-worker	Simula procesamiento de órdenes
mongodb	Persistencia de datos
rabbitmq	Cola de órdenes
redis	Se agregará como extensión del laboratorio

Narrativa pedagógica

El flujo base será:

```

store-front / virtual-customer
  |
  v
order-service
  |
  v
RabbitMQ queue: orders
  |
  v
makeline-service
  |
  v
MongoDB

```

Mensaje clave:

La orden no necesariamente se completa en una sola transacción. Se recibe, se encola, se procesa después y eventualmente llega a un estado final.

0. Preparación del entorno en Azure Cloud Shell

0.1 Abrir Azure Cloud Shell

Desde Azure Portal:

1. Abrir **Cloud Shell**
 2. Seleccionar **Bash**
 3. Confirmar que tienes acceso al cluster AKS
-

0.2 Variables del laboratorio

Ajusta los valores según tu entorno:

```
export RG_NAME="<TU_RESOURCE_GROUP>"
export AKS_NAME="<TU_AKS_NAME>"
export NS="aks-store-state-lab"
```

Ejemplo:

```
export RG_NAME="rg-microservices-lab"
export AKS_NAME="aks-microservices-lab"
export NS="aks-store-state-lab"
```

0.3 Conectarse al AKS

```
az aks get-credentials \
  --resource-group $RG_NAME \
  --name $AKS_NAME \
  --overwrite-existing
```

Validar conexión:


```
kubectl get nodes
kubectl get ns
```

Resultado esperado:

NAME	STATUS	ROLES
aks-nodepool1-xxxxxx	Ready	agent

0.4 Crear namespace

```
kubectl create namespace $NS
kubectl config set-context --current --namespace=$NS
```

Validar:

```
kubectl get ns
kubectl config view --minify | grep namespace
```

1. Descargar y preparar AKS Store Demo

1.1 Descargar manifiesto oficial

```
curl -L \
  https://raw.githubusercontent.com/Azure-Samples/aks-store-demo/main/aks-
  store-all-in-one.yaml \
  -o aks-store-all-in-one.yaml
```

1.2 Evitar LoadBalancers públicos para el laboratorio

El manifiesto original expone `store-front` y `store-admin` como `LoadBalancer`.

Para un laboratorio controlado, usaremos `ClusterIP` y `port-forward`.

```
sed -i 's/type: LoadBalancer/type: ClusterIP/g' aks-store-all-in-one.yaml
```

Validar:

```
grep -n "type:" aks-store-all-in-one.yaml | head
```

1.3 Desplegar aplicación

```
kubectl apply -f aks-store-all-in-one.yaml -n $NS
```

Ver recursos:

```
kubectl get all -n $NS
```

Esperar a que todo esté listo:

```
kubectl wait --for=condition=Ready pod --all -n $NS --timeout=300s
```

1.4 Validar deployments y statefulsets

```
kubectl get deploy -n $NS  
kubectl get statefulset -n $NS  
kubectl get svc -n $NS
```

Resultado esperado:

```
deployment.apps/order-service  
deployment.apps/product-service  
deployment.apps/makeline-service  
deployment.apps/store-front  
deployment.apps/store-admin  
deployment.apps/virtual-customer  
deployment.apps/virtual-worker  
  
statefulset.apps/mongodb  
statefulset.apps/rabbitmq
```

2. Explorar arquitectura de microservicios

2.1 Ver pods

```
kubectl get pods -n $NS -o wide
```

Pregunta para estudiantes:

¿Qué servicios son stateless y cuáles tienen estado?

Respuesta esperada:

Recurso	Tipo
store-front	Stateless
store-admin	Stateless
order-service	Stateless
product-service	Stateless
makeline-service	Stateless
virtual-customer	Stateless
virtual-worker	Stateless
mongodb	Stateful
rabbitmq	Stateful

2.2 Ver servicios internos

```
kubectl get svc -n $NS
```

Explicar:

- `ClusterIP` permite comunicación interna
 - No todos los servicios deben estar expuestos públicamente
 - Los frontends se pueden abrir temporalmente con `port-forward`
-

2.3 Acceder a store-front

En una terminal:

```
kubectl port-forward svc/store-front 8080:80 -n $NS
```

Abrir en navegador:

```
http://localhost:8080
```

En Cloud Shell, usar **Web Preview** sobre el puerto `8080`.

2.4 Acceder a store-admin

En otra terminal:

```
kubectl port-forward svc/store-admin 8081:80 -n $NS
```

Abrir:

```
http://localhost:8081
```

En Cloud Shell, usar **Web Preview** sobre el puerto `8081`.

2.5 Ver logs iniciales

```
kubectl logs deploy/order-service -n $NS --tail=50  
kubectl logs deploy/makeline-service -n $NS --tail=50  
kubectl logs deploy/product-service -n $NS --tail=50
```

Mensaje clave:

En microservicios, los logs son parte de la arquitectura. Si no puedes seguir una operación, no puedes operar el sistema.

3. Event-driven con RabbitMQ

3.1 Acceder a RabbitMQ Management UI

```
kubectl port-forward svc/rabbitmq 15672:15672 -n $NS
```

Abrir:

```
http://localhost:15672
```

Credenciales del manifiesto:

```
username: username  
password: password
```

3.2 Observar la cola `orders`

En RabbitMQ UI:

1. Ir a **Queues and Streams**
2. Buscar la cola `orders`
3. Observar:
 - Ready
 - Unacked
 - Total
 - Consumers

3.3 Generar actividad

La aplicación ya incluye `virtual-customer`, que genera órdenes automáticamente.

Ver logs:

```
kubectl logs deploy/virtual-customer -n $NS --tail=100
kubectl logs deploy/order-service -n $NS --tail=100
kubectl logs deploy/makeline-service -n $NS --tail=100
```

3.4 Explicación

Flujo conceptual:

```
virtual-customer
  |
  v
order-service
  |
  v
RabbitMQ orders queue
  |
  v
makeline-service
  |
  v
mongodb
```

Mensaje clave:

RabbitMQ desacopla la recepción de la orden del procesamiento final. Esto mejora disponibilidad, pero introduce consistencia eventual.

3.5 Pregunta de discusión

Si `order-service` acepta la orden pero `makeline-service` aún no la procesa, ¿la orden existe o no existe?

Respuesta guiada:

- Existe como intención o mensaje en cola
- Puede no existir todavía como orden final procesada
- El sistema está en estado intermedio
- Esto es consistencia eventual

4. Fallo parcial: detener makeline-service

4.1 Escenario

Vamos a detener el servicio que procesa órdenes desde RabbitMQ.

Esto simula:

- Worker caído
 - Consumidor indisponible
 - Backlog de mensajes
 - Eventual consistency visible
-

4.2 Escalar makeline-service a cero

```
kubectl scale deployment makeline-service --replicas=0 -n $NS
```

Validar:

```
kubectl get pods -n $NS  
kubectl get deploy makeline-service -n $NS
```

4.3 Observar RabbitMQ

En RabbitMQ UI:

1. Ir a la cola `orders`
2. Observar cómo los mensajes pueden acumularse
3. Revisar número de consumers

También ver logs:

```
kubectll logs deploy/order-service -n $NS --tail=100
kubectll logs deploy/virtual-customer -n $NS --tail=100
```

4.4 Explicación

¿Qué está pasando?

```
order-service sigue aceptando órdenes
RabbitMQ guarda mensajes
makeline-service no consume
MongoDB no recibe nuevas órdenes procesadas
```

Mensaje clave:

El sistema sigue parcialmente disponible, pero el estado final se retrasa.

4.5 Discusión CAP

Pregunta:

¿Este comportamiento favorece disponibilidad o consistencia fuerte?

Respuesta:

- Favorece disponibilidad
- Acepta inconsistencia temporal
- Es un enfoque más cercano a AP
- No bloquea todo el sistema por caída de un consumidor

4.6 Restaurar makeline-service

```
kubectl scale deployment makeline-service --replicas=1 -n $NS
kubectl wait --for=condition=Ready pod -l app=makeline-service -n $NS --
    timeout=180s
```

Ver logs:

```
kubectl logs deploy/makeline-service -n $NS --tail=100
```

Observar RabbitMQ:

- Mensajes consumidos
- Cola reducida
- Flujo recuperado

4.7 Mensaje instructor

Este es el valor de un broker: desacopla temporalmente productor y consumidor. Pero no elimina la complejidad: ahora hay que gestionar backlog, reintentos, duplicados e idempotencia.

5. Fallo parcial: detener RabbitMQ

5.1 Escenario

Ahora detenemos el broker.

Esto simula una falla más crítica:

```
order-service quiere publicar evento
RabbitMQ no está disponible
mensaje no puede encolarse
```

5.2 Escalar RabbitMQ a cero

RabbitMQ está desplegado como StatefulSet.

```
kubectl scale statefulset rabbitmq --replicas=0 -n $NS
```

Validar:

```
kubectl get pods -n $NS
kubectl get statefulset rabbitmq -n $NS
```

5.3 Revisar comportamiento

Ver logs de `order-service`:

```
kubectl logs deploy/order-service -n $NS --tail=100
```

Ver eventos de Kubernetes:

```
kubectl get events -n $NS --sort-by=.lastTimestamp | tail -30
```

5.4 Pregunta para estudiantes

Si el broker está caído, ¿qué debe hacer `order-service` ?

Opciones:

Opción A: Rechazar la orden

Ventaja:

- No se pierde intención de negocio
- Consistencia más fuerte

Desventaja:

- Menor disponibilidad
- Peor experiencia del cliente

Opción B: Aceptar y guardar localmente

Ventaja:

- Mayor disponibilidad
- Mejor experiencia

Desventaja:

- Necesita Outbox Pattern
- Necesita reintentos
- Necesita reconciliación

5.5 Explicación: Outbox Pattern

Problema:

Guardar orden en DB y publicar evento en RabbitMQ no es una transacción atómica.

Solución conceptual:

1. Guardar orden en DB
2. Guardar evento en tabla outbox

3. Worker lee outbox
4. Publica a RabbitMQ
5. Marca evento como publicado

Diagrama:

```
Order Service
|
+--> Orders DB
|
+--> Outbox Table
      |
      v
    Outbox Publisher
      |
      v
    RabbitMQ
```

Mensaje clave:

El Outbox Pattern evita perder eventos cuando el broker está temporalmente caído.

5.6 Restaurar RabbitMQ

```
kubectl scale statefulset rabbitmq --replicas=1 -n $NS
kubectl wait --for=condition=Ready pod -l app=rabbitmq -n $NS --timeout=180s
```

Validar:

```
kubectl get pods -n $NS
```

6. Fallo parcial: detener MongoDB

6.1 Escenario

MongoDB es el almacenamiento persistente del demo.

Detenerlo permite explicar:

- dependencia de persistencia
 - degradación funcional
 - diferencia entre stateless y stateful
 - necesidad de resiliencia en capa de datos
-

6.2 Escalar MongoDB a cero

MongoDB también está como StatefulSet.

```
kubectl scale statefulset mongodb --replicas=0 -n $NS
```

Validar:

```
kubectl get pods -n $NS  
kubectl get statefulset mongodb -n $NS
```

6.3 Probar aplicación

Intentar:

- abrir `store-front`
- abrir `store-admin`
- listar productos
- observar errores

Ver logs:

```
kubectll logs deploy/product-service -n $NS --tail=100
kubectll logs deploy/makeline-service -n $NS --tail=100
kubectll logs deploy/store-admin -n $NS --tail=100
```

6.4 Discusión

Pregunta:

¿Qué operaciones deberían seguir funcionando si MongoDB cae?

Posibles respuestas:

- Frontend puede cargar parcialmente
- Catálogo puede fallar si depende de DB
- Nuevas órdenes pueden fallar si procesamiento requiere persistencia
- Si hay cache, algunas lecturas podrían sobrevivir

6.5 Conexión con resiliencia

Conceptos:

- Circuit breaker
- Timeout
- Retry con backoff
- Fallback
- Cache como degradación
- Read models

Mensaje clave:

Cuando la base de datos cae, la pregunta no es solo técnica. Es de negocio: ¿qué experiencia degradada aceptamos?

6.6 Restaurar MongoDB

```
kubectl scale statefulset mongodb --replicas=1 -n $NS  
kubectl wait --for=condition=Ready pod -l app=mongodb -n $NS --timeout=180s
```

Validar:

```
kubectl get pods -n $NS
```

7. Redis Cache Lab

7.1 Objetivo

Agregar Redis para demostrar:

- Cache distribuido
 - TTL
 - Cache Aside
 - Stale data
 - Invalidación manual
-

7.2 Desplegar Redis

```
cat <<'EOF' > redis-lab.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: redis
  namespace: aks-store-state-lab
spec:
  replicas: 1
  selector:
    matchLabels:
      app: redis
  template:
    metadata:
      labels:
        app: redis
    spec:
      nodeSelector:
        "kubernetes.io/os": linux
      containers:
        - name: redis
          image: redis:7
          ports:
            - containerPort: 6379
          resources:
            requests:
              cpu: 5m
              memory: 32Mi
            limits:
              cpu: 100m
              memory: 128Mi
---
apiVersion: v1
kind: Service
metadata:
```

```
name: redis
namespace: aks-store-state-lab
spec:
  selector:
    app: redis
  ports:
    - port: 6379
      targetPort: 6379
EOF
```

Si usaste otro namespace, reemplaza `aks-store-state-lab`:

```
sed -i "s/namespace: aks-store-state-lab/namespace: $NS/g" redis-lab.yaml
```

Aplicar:

```
kubectl apply -f redis-lab.yaml -n $NS
kubectl wait --for=condition=Ready pod -l app=redis -n $NS --timeout=120s
```

7.3 Entrar a Redis

```
REDIS_POD=$(kubectl get pod -n $NS -l app=redis -o
  jsonpath='{.items[0].metadata.name}')
kubectl exec -it $REDIS_POD -n $NS -- redis-cli
```

7.4 Crear producto en cache

Dentro de Redis:

```
SET product:dog-food '{"productId":"dog-food","name":"Dog
  Food","price":25.00,"stock":100}'
GET product:dog-food
TTL product:dog-food
```

Agregar TTL:

```
EXPIRE product:dog-food 300
TTL product:dog-food
```

Salir:

```
exit
```

7.5 Explicar Cache Aside

Patrón:

1. Cliente pide producto
2. Servicio consulta Redis
3. Si existe: devuelve desde cache
4. Si no existe: consulta DB
5. Guarda resultado en Redis
6. Devuelve respuesta

Diagrama:

```
Client
|
v
Product Service
|
+--> Redis
|
+-- HIT --> return cached product
|
+-- MISS --> MongoDB --> Redis --> return product
```

7.6 Simular stale data

Ver valor actual:

```
kubectl exec -it $REDIS_POD -n $NS -- redis-cli GET product:dog-food
```

Ahora simular que el precio real cambió a 20.00, pero Redis sigue con 25.00.

Discusión:

¿Qué pasa si el cliente compra con precio viejo? ¿El precio del catálogo puede ser eventualmente consistente? ¿El precio de checkout puede ser cacheado?

7.7 Invalidar cache

```
kubectl exec -it $REDIS_POD -n $NS -- redis-cli DEL product:dog-food
kubectl exec -it $REDIS_POD -n $NS -- redis-cli GET product:dog-food
```

Resultado esperado:

```
(nil)
```

7.8 Recrear cache con valor actualizado

```
kubectl exec -it $REDIS_POD -n $NS -- redis-cli SET product:dog-food
'{"productId":"dog-food","name":"Dog Food","price":20.00,"stock":100}'
kubectl exec -it $REDIS_POD -n $NS -- redis-cli EXPIRE product:dog-food 300
kubectl exec -it $REDIS_POD -n $NS -- redis-cli GET product:dog-food
```

7.9 Discusión de estrategias

Estrategia	Descripción	Ventaja	Riesgo
Cache Aside	App lee cache, si miss lee DB	Simple	Stale data
Write Through	Escribe cache y DB juntos	Cache actualizado	Escrituras lentas
Write Behind	Escribe cache primero, DB después	Rápido	Riesgo de pérdida
TTL Only	Expira por tiempo	Fácil	Ventana de inconsistencia

8. CAP Theorem aplicado al laboratorio

8.1 Caso 1: makeline-service caído

```
RabbitMQ disponible  
order-service disponible  
makeline-service caído
```

Decisión:

- Se aceptan órdenes
- Se acumulan mensajes
- Se procesa después

Interpretación:

- Mayor disponibilidad
 - Consistencia eventual
-

8.2 Caso 2: RabbitMQ caído

```
order-service disponible  
RabbitMQ caído
```

Decisión posible:

- Rechazar órdenes
- O guardar intención localmente con Outbox

Interpretación:

- Si rechazo: más CP
 - Si acepto con Outbox: más AP, pero más complejidad
-

8.3 Caso 3: MongoDB caído

product-service / makeline-service dependen de MongoDB

Decisión posible:

- Fallar rápido
- Responder desde cache
- Activar modo degradado

Interpretación:

- Cache mejora disponibilidad
 - Pero puede entregar datos stale
-

9. Saga conceptual sobre AKS Store Demo

AKS Store Demo no implementa una Saga completa de e-commerce con pagos e inventario separados, pero sí permite explicar el concepto usando su flujo real.

9.1 Flujo actual

```
graph TD; A[Order Created] --> B[Order Queued]; B --> C[Order Processed];
```

9.2 Flujo Saga extendido conceptual

1. Create Order
2. Reserve Inventory
3. Process Payment
4. Create Shipment
5. Confirm Order

9.3 Compensaciones

Acción	Compensación
Crear orden	Cancelar orden
Reservar inventario	Liberar inventario
Procesar pago	Reembolsar pago
Crear envío	Cancelar envío

9.4 Mensaje clave

En microservicios, el rollback no siempre es técnico. Muchas veces es una nueva acción de negocio que compensa el efecto anterior.

10. Observabilidad básica

10.1 Ver logs por servicio

```
kubectl logs deploy/order-service -n $NS --tail=100
kubectl logs deploy/makeline-service -n $NS --tail=100
kubectl logs deploy/product-service -n $NS --tail=100
kubectl logs deploy/store-front -n $NS --tail=100
```

10.2 Seguir logs en vivo

```
kubectl logs deploy/order-service -n $NS -f
```

En otra terminal:

```
kubectl logs deploy/makeline-service -n $NS -f
```

10.3 Ver eventos de Kubernetes

```
kubectl get events -n $NS --sort-by=.lastTimestamp
```

10.4 Ver estado de pods

```
kubectl get pods -n $NS -o wide
kubectl describe pod -n $NS <POD_NAME>
```

10.5 Mensaje clave

Sin observabilidad, un sistema distribuido es una caja negra. Ver pods corriendo no significa entender el estado de negocio.

11. Ejercicio para estudiantes

Escenario

El negocio pide que la tienda siga aceptando órdenes aunque `makeline-service` esté caído durante 10 minutos.

Preguntas

1. ¿Qué componente absorbe el fallo?
 2. ¿Qué métrica deberíamos monitorear?
 3. ¿Cuándo deberíamos dejar de aceptar órdenes?
 4. ¿Qué pasa si RabbitMQ también cae?
 5. ¿Necesitamos Outbox Pattern?
 6. ¿Qué operación debe ser idempotente?
 7. ¿Dónde habría que poner correlationId?
-

Respuesta esperada

1. RabbitMQ absorbe temporalmente el fallo.
 2. Longitud de la cola, tasa de consumo, edad del mensaje más viejo.
 3. Cuando el backlog supera un umbral de negocio.
 4. El sistema debe rechazar o usar Outbox.
 5. Sí, si queremos aceptar órdenes aunque el broker esté caído.
 6. Procesamiento de orden y publicación de eventos.
 7. Desde la entrada en `store-front/order-service` hasta el evento en RabbitMQ y el procesamiento en `makeline-service`.
-

12. Checklist arquitectónico

Antes de diseñar persistencia distribuida, validar:

- ¿Quién es dueño del dato?
 - ¿Existe shared database?
 - ¿Hay comunicación síncrona innecesaria?
 - ¿Qué pasa si el consumidor cae?
 - ¿Qué pasa si el broker cae?
 - ¿Qué pasa si la DB cae?
 - ¿Hay idempotencia?
 - ¿Hay compensación?
 - ¿Hay timeout?
 - ¿Hay retry con backoff?
 - ¿Hay correlationId?
 - ¿Qué inconsistencia tolera el negocio?
 - ¿Qué datos se pueden cachear?
 - ¿Cómo se invalida cache?
 - ¿Qué métricas indican degradación?
-

13. Limpieza del laboratorio

13.1 Eliminar Redis

```
kubectl delete -f redis-lab.yaml -n $NS
```

13.2 Eliminar AKS Store Demo

```
kubectl delete -f aks-store-all-in-one.yaml -n $NS
```

13.3 Eliminar namespace

```
kubectl delete namespace $NS
```

Validar:

```
kubectl get ns
```

14. Cierre para estudiantes

En este laboratorio vimos una aplicación real en AKS y la usamos para estudiar problemas de arquitectura distribuida.

Conceptos clave:

- La persistencia ya no es solo “qué base de datos usar”.
- La pregunta real es quién es dueño del estado.
- RabbitMQ desacopla, pero introduce consistencia eventual.
- MongoDB centraliza persistencia en este demo, pero en un diseño más avanzado cada bounded context tendría su propia base.
- Redis mejora performance, pero puede devolver datos stale.
- Cuando un servicio cae, el sistema debe decidir entre consistencia y disponibilidad.
- Sagas no eliminan fallos: los hacen manejables con compensaciones.
- Sin observabilidad, no hay operación real de microservicios.

Mensaje final:

En microservicios, diseñar persistencia significa diseñar comportamiento bajo fallo.